**SYSTEMS AND METHODS FOR BIG AND UNSTRUCTURED DATA**
**Prof. Marco Brambilla**

POLITECNICO MILANO 1863

**January 18, 2022**

Last Name _____ First Name _____ Codice Persona _____

## Theory (6 PT)

Choose whether the following statements are true or false by marking the CORRECT answers with an X.
Each correct answer awards 0,5 PT while each incorrect answer awards -0,25 PT.

| | | | |
|---|---|---|---|
| T | *Pig* is high-level language for data analysis used to express sequences of MapReduce jobs. | **True** | |
| F | All APIs require the user to authenticate through the OAuth protocol. | | **False** |
| T | The data wrangling process consists of four steps: understand, cleanse, augment, and shape. | **True** | |
| T | Both replication and partitioning are needed to grant proper management of data distribution. | **True** | |
| F | *Vertical replication* is appropriate when a business deals with flexible and increasing size of (big) datasets | **True** | **False** |
| T | *MongoDB* supports SSL both between client and server and between clusters. | **True** | |
| F | *Neo4J* implements *BASE* transactions. | | **False** |
| F | The main motivation of gossip protocols is that of increasing the alignment between the different nodes in a data mgmt. cluster | | **False** |
| F | Prescriptive analysis is a type of analysis that explains what happened in the past by looking at the data. | | **False** |
| F | In Cassandra, a partition key defines how the data is stored within a partition. | | **False** |
| F | MongoDB is mainly focused on Consistency and Availability. | | **False** |
| T | Depending on business scenarios, data properties, and size, one selects the appropriate data models and technology mix. | **True** | |

**SYSTEMS AND METHODS FOR BIG AND UNSTRUCTURED DATA**

**Prof. Marco Brambilla**

POLITECNICO MILANO 1863

**January 17, 2022**

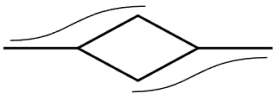
Last Name _____ First Name _____ Codice Persona _____

For each question, one correct answer is provided. There could be more!

Suppose you need a data storage solution for supporting the information system of a media production company. The company needs to track the production process of media products (shows of various types: TV programs, series, movies). Each show has a category (comedy, drama, etc.). The company keeps track of all its shows and of the people on its payroll. Each person has their personal data, bio, and detailed history of their activities with the company and other companies: the shows they worked in, their role, their paychecks, the period of work covered (date from-to). Each show has a title, description, production year, list of reviews, list of locations where the show was shot and is linked to all the people working at it. Both shows and people may win some awards. These are recorded too (type and date). Shows are also described by the various expenses recorded (equipment, services, rentals, and so on), with date, type of expense, invoice, amount and name of the company providing the service or product. For each show, all events are recorded in detail to keep track of the advancement of the production (each step of the writing, casting, shooting, contracts signed, and so on). Events may amount to tens per day. Shows have a script, which consists of the complete writing of the show's organisation (i.e., what it's expected to happen, and how; which actor says what and who does what; and so on). The company also includes staff workers not associated with shows. Information about them is also available.

**Exercise 1 (3 PT)**

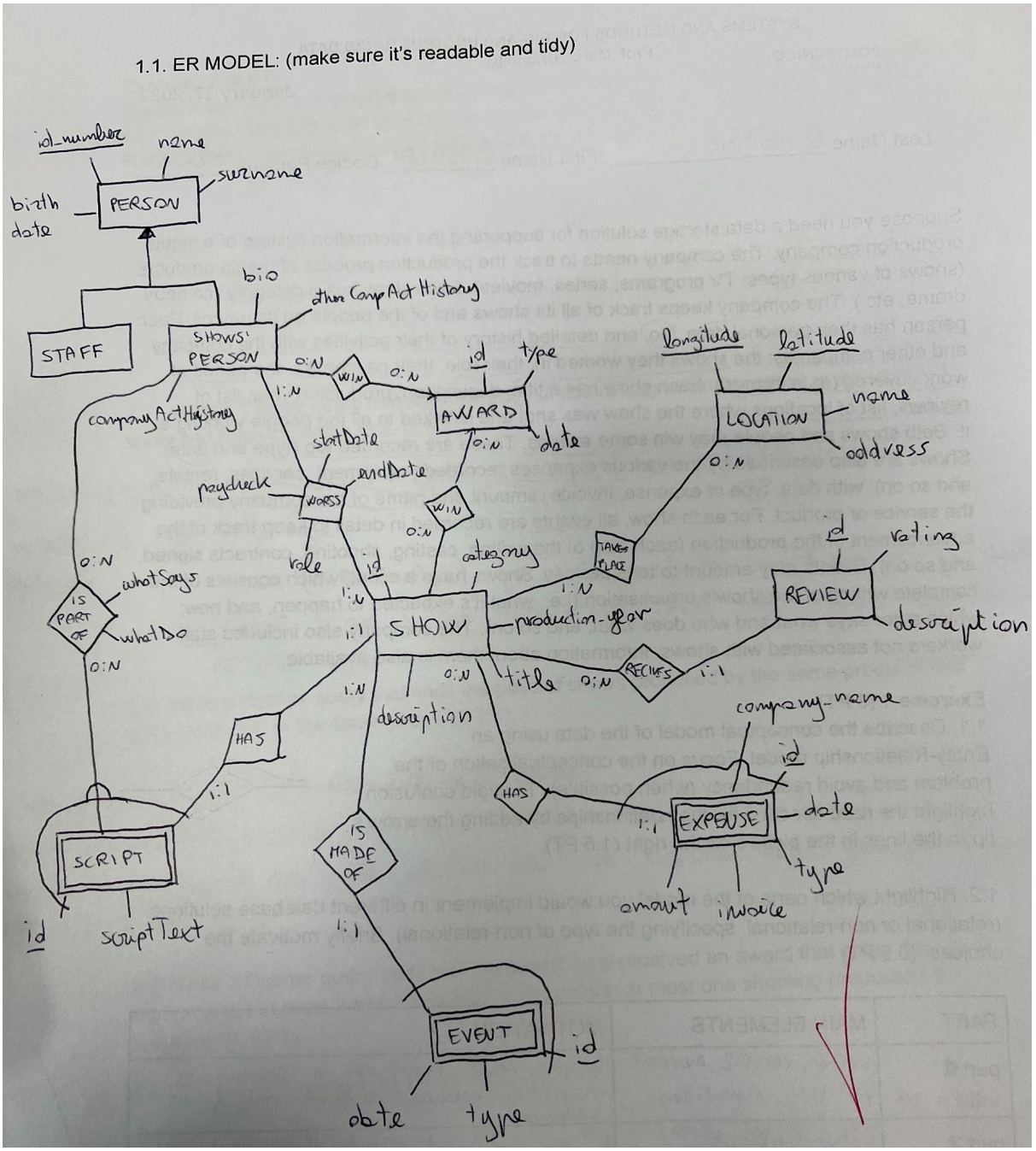
1.1. Describe the conceptual model of the data using an Entity-Relationship model. Focus on the conceptualisation of the problem and avoid redundancy (when possible). To avoid confusion, highlight the read direction for the relationships by adding the arrow's tip to the lines in the picture on the right (1,5 PT).



1.2. Highlight which parts of the model you would implement in different database solutions (relational or non-relational, specifying the type of non-relational). Briefly motivate the choices. (0,5 PT)

| DB | MAIN ELEMENTS | MOTIVATION |
|---|---|---|
| Graph | Show, People, Awards, Location, Interactions | Useful to research the relationships of the different parts of the schema |
| Documental | Show Staff Members, Show, Award, Activity | Useful to store all the elements together, flexible structure |
| Elasticsearch | Script, People, Histories | Provides Text search and text analysis functionalities |

1.1. ER MODEL: (make sure it's readable and tidy)



1.3 Represent a graph schema for at least four different nodes, their respective relations and their attributes that you would implement in a graph DB. Be coherent with the names you provided in the ER diagram. (0,5 PT)

Simple Request

1.4 Provide an example of a document for the *Show* entity and the entities it contains. For each entity, provide its attributes at least once within the document. Be coherent with the name you provided in your ER diagram and the data type. (0,5 PT)

Simple Request

**Exercise 2 (4 PT)**
Suppose you store in a graph database in Neo4j the lists of people, shows, and their connections. We advise you to provide a simple representation of the graph database if you haven't already done it during the 1st Exercise. Be coherent with your ER diagram.

2.1. Write a Cypher query that finds the pairs of actors who acted (together) in the same show and have the same age. (1 PT)

```
MATCH (a: showperson) – [r: work_in] → (s: show) ← [t: work_in] – (b: showperson)
WHERE a.age = b.age AND r.role = "actor" AND t.role = "actor" AND a.id <> b.id
RETURN a, b
```

2.2. Write a Cypher query that finds the pairs of shows produced by the same producer and were produced in the same year. (1,5 PT)

```
MATCH (s: show) ← [r: work_in] – (a: showperson) – [t: work_in] → (b: show)
WHERE b.id <> s.id AND r.role = "director" AND t.role = "director" AND
        s.prod_year = b.prod_year
RETURN s, b
```

2.3. Write a Cypher query that finds the people who received an award that acted in a show that received at least 2 awards in 2021 and in which at most one shooting production is included. (1,5 PT)

```
MATCH (p: showperson) – [: wins] → [: award],
      (p) – [: work_in {role: "actor"}] – (s: show) – [: win] – (awd: award {year: "2021"}),
      (s) – [: has] – (e: event {type: "shooting production")
WITH p, count(awd) as awd_count, count(e) as shooting_count
WHERE award_count > 1 AND shooting_count < 2
RETURN DISTINCT p
```

Last Name _____ First Name _____ Codice Persona _____

**Exercise 3 (3 PT)**
Suppose you store in a documental database in MongoDB the lists of shows, awards, locations and reviews and their connections. We advise you to provide a simple document representation if you haven't already done it during the 1st Exercise. Write the whole query, naming your collection. Be coherent with your ER diagram.

3.1. Write a query to collect 5 shows produced in 2021 in which at least one of the expenses is of type "services". (1,5 PT)

```
db.shows.find({ "$and": [{"production_year": "2021"}, {"expenses.type": "services"}])
        .limit(5)
```

3.2. Write a query to collect the number of awards received by each show whose average review vote/rating is greater than 4. (1,5 PT)

```
db.shows.aggregate([
    {"$group": {
        "_id": true,
        "avg_rating": {"$avg": "reviews.rating"},
        "awards": {"$size": "awards"}
    }},
    {"$match": {"avg_rating": {"$gt": 4}}
}
```

**Exercise 4 (3 PT)**
Suppose you store an Elasticsearch index of the reviews. Include an attribute representing the show's title (i.e., a sort of an “external key”). Be coherent with your ER diagram. Write the whole query for each of the cases below, naming your index.

4.1. Provide the complete mapping of the reviews index (i.e., field name, field type, the structure of the mapping, etc.) (0,5 PT)

```
PUT reviews
{
	“mappings”: {
		“properties”: {
			“description”: {
				“type”: “text”
			}
			…
		}
	}
}
```

4.2. Write a query to extract the count of positive reviews (i.e., vote/rating greater or equal to 4) received by each show (1 PT)

```
GET /reviews/_search
{
	“size”: 0,
	“query”: {
		“bool”: {
			“must”: {
				“range”: {
					“rating”: {“gte”: 4}
				}
			}
		}
	},
	“aggs”: {
		“positive_ratings”: {
			“terms”: {
				“field”: “title”
			}
		}
	}
}
```

4.3. Write a query to extract the list of all the reviews whose text contains the word “good”, ordered by Elasticsearch return score. (1,5 PT)

```
GET /_search
{
        "query": {
                "match": {
                        "description": “good” → The field type must be text (uses an analyzer)
                }
        }
}
```

**Exercise 5 (3 PT)**

While evaluating this exercise, no points were detracted if you identified incorrect errors. Instead, you were awarded points for each of the mistakes (among the ones in the solution provided) you found and fixed.

The media production company has employed you to manage a Cassandra database. A colleague of yours provides you with a Cassandra script (i.e., a series of operations) to set up the database. The operations included in the script are

- the creation of a keyspace named “staff_management_system”,
- the use of the keyspace named “staff_management_system”,
- the creation of a table named “staff_members”,
- the manual insertion of a tuple to test the database,
- the upload of a list of tuples from a file named “staff_members.txt”,
- execution of two queries as a final test.

Your task is to check that the flow of operations has been properly coded. If **any** mistake is found, mark it and provide the solution in the space provided by re-writing the command or describing how to fix it. There could be no errors, or one or multiple errors in each step. Errors can be of multiple types.

```
CREATE KEYSPACE “staff_management_system” WITH replication =
{‘class’: ‘SimpleStrategy’, ‘replication_factor’: 15};
```

No errors (the apices in the name of the table are accepted)

```
USE staff_management_system;
```

No Errors

```
CREATE TABLE staff_members (
      name string,
      surname string,
      personal_id string,
      age varint,
      PRIMARY KEY (personal_id)
);
```

**Error**: The type string does not exist in Cassandra
**Solution**: Use the type text (or varchar)

```
INSERT INTO staff_members (name, personal_id, surname, age) VALUES
(“Francesco”, FRNRSS95E12F675T, “Rossi”, 19);
```

**Error**: Type mismatch in the personal_id field
**Solution**: Add apices to match the text type

```
UPLOAD “D:/Program Files/Cassandra/Data/staff_members.txt”;
```

**Error**: The UPLOAD command does not exist in Cassandra
**Solution**: The correct command to performed the mentioned operation is SOURCE

```
SELECT name, surname
FROM staff_management_system
WHERE personal_id = “FRNTCC95E12F675T”;
```

**Error**: Both this query and the following one are performed on the keyspace “staff_management_system” rather than the table “staff_members”
**Solution**: Change the table to “staff_members”

```
SELECT COUNT(*)
FROM staff_management_system
WHERE name = “Francesco”;
```

**Error**: There is no clustering key, or secondary index for the field name
**Solution**: Either create an index for the name before the query, add a clustering key on name when creating the table, or use ALLOW FILTERING

……